Arch Technologies

Data Science Internship — Month 1 Report

Titanic Survival Classification • Stock Price Prediction

Full Name	Muhammad Hashir
Internship Domain	Data Science
Email Address	mhkh3200@gmail.com
Phone Number	+92 371 0971465
Organisation	Arch Technologies
Report Period	Month 1 — November 2026
Submission	submissions.archtech@gmail.com

This report shows my two Month 1 tasks in full: the data I used, how I did the work, my code, my results with charts, and what I found. Every number in this report comes from code I actually ran. My full code, data and results are in the GitHub repos that go with this report.

My work: github.com/mhkh-stock/titanic-survival-classification • github.com/mhkh-stock/aapl-stock-price-prediction

Introduction

Data science means finding useful things in data. I use statistics, code and machine learning to find patterns in big data and guess what will happen next. Companies use it in health, finance, marketing and business. Google and Amazon use it to make their services better.

I did two tasks in Month 1 of my Data Science internship at Arch Technologies:

1. Titanic Survival Classification.

2. Stock Price Prediction (Apple stock).

I did both tasks from start to finish. I got real data from public sources, cleaned it, trained models and checked them on data the models had never seen. I used Python, pandas, scikit-learn, matplotlib and NumPy.

Task 1 — Titanic Survival Classification

My goal

I wanted to build a model that can guess if a passenger on the Titanic survived or not. I used age, sex, ticket class and fare.

The data

I used the Kaggle Titanic training set: 891 passengers and 12 columns. I got it from https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv (a public copy of the Kaggle train.csv file. I checked it after I downloaded it: 891 rows and 12 columns). In total 342 passengers survived (38.4%) and 549 did not.

Passenger class	Survival rate
Class 1	62.96%
Class 2	47.28%
Class 3	24.24%

Sex	Survival rate
Female	74.20%
Male	18.89%

Two things stood out before I built any model. Women survived much more than men (74.20% vs 18.89%). And first-class passengers survived much more than third-class

passengers (62.96% vs 24.24%). Age and fare mattered too. The feature chart at the end of this part shows it.

How I cleaned the data

I wrote down each cleaning step and why I did it:

Problem	What I did	Why
Cabin — 687 of 891 missing (77%)	I dropped the column	Too much was missing, so it had no real signal
Age — 177 missing (19.9%)	I filled each one with the median age of the passenger's Sex and Ticket Class group	This keeps every row, and the group median is a better guess than one median for all
Embarked — 2 missing	I filled them with S	S is the most common port, and it was only two rows
Fare — 0 missing	I changed nothing	The column was complete

Then I made new features. I took the title from the Name (Mr, Mrs, Miss, Master, or Rare for the uncommon ones). I made FamilySize = SibSp + Parch + 1, and an IsAlone flag. I changed Sex to 1 for female and 0 for male. I one-hot encoded Embarked and Title. I saved the cleaned data (891 rows, 14 features) as data/titanic_clean.csv.

I split the data into a training set (712 passengers, 80%) and a test set (179 passengers, 20%). I kept the same 38.4% survival rate in both sets, and I used seed 42 so anyone can get the same result.

My code

My full code is in src/titanic.py in the repo. Here are the main parts.

Cleaning and making new features:

```
# Cabin: 77% missing -> drop the column
df = df.drop(columns=["Cabin"])
# Age: fill with the median Age of the passenger's Sex x Pclass group
df["Age"] = df.groupby(["Sex", "Pclass"])["Age"].transform(
    lambda s: s.fillna(s.median()))
# Embarked: 2 missing -> most common port, Southampton ('S')
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# Feature engineering
df["Title"] = df["Name"].str.extract(r",\s*([A-Za-z]+)\.", expand=False)
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
```

Splitting the data and training two models:

```
X_train, X_test, y_train, y_test = train_test_split(
    features, target, test_size=0.20, random_state=42,
```

```
    stratify=target)

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(StandardScaler().fit_transform(X_train), y_train)

rf = RandomForestClassifier(n_estimators=300, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
```

Checking the models — accuracy, precision, recall, F1 and ROC-AUC, all on the 179 test passengers the models never saw in training:

```
accuracy_score(y_test, pred)
precision_score(y_test, pred)
recall_score(y_test, pred)
f1_score(y_test, pred)
roc_auc_score(y_test, proba)
```

My results

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.8547	0.8413	0.7681	0.8030	0.8758
Random Forest	0.8101	0.7612	0.7391	0.7500	0.8266

Logistic Regression did better. It was best on every score. I learned from this that a simple model can beat a complex one when the data is small. Random Forest was close behind. Logistic Regression's confusion matrix shows 100 right 'died' guesses and 53 right 'survived' guesses out of 179 test passengers. Random Forest's shows 94 right 'died' guesses and 51 right 'survived' guesses.

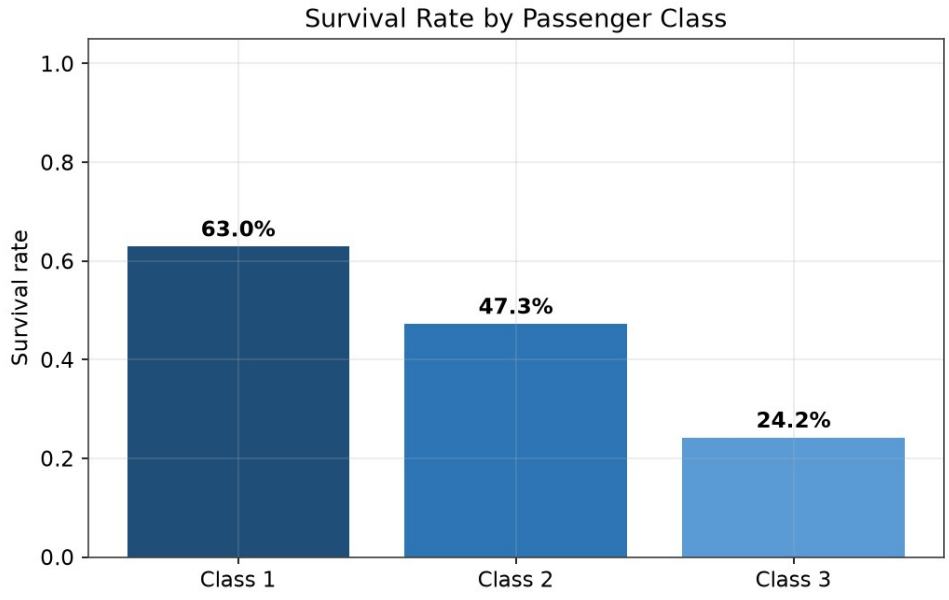


Figure 1 — Survival rate by passenger class.

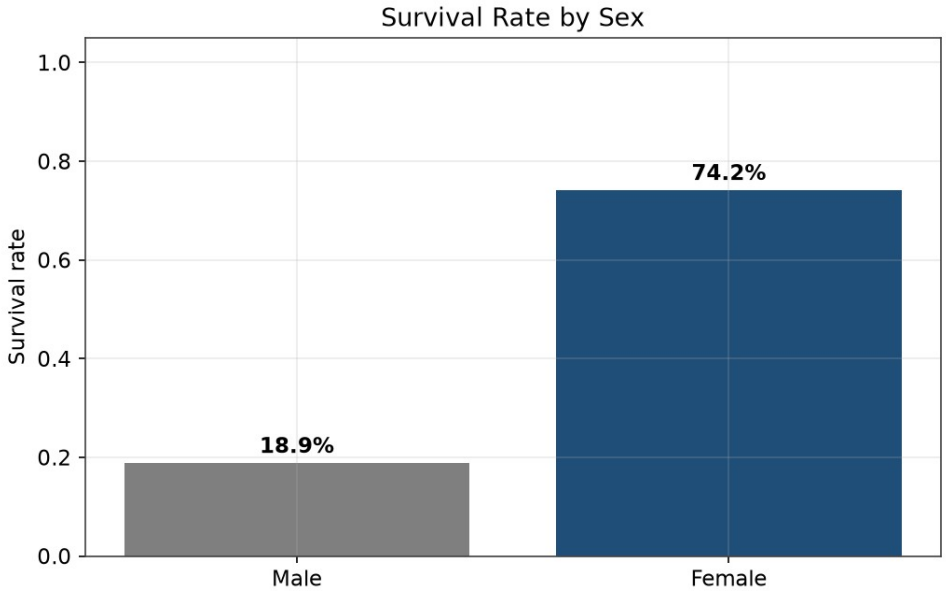


Figure 2 — Survival rate by sex.

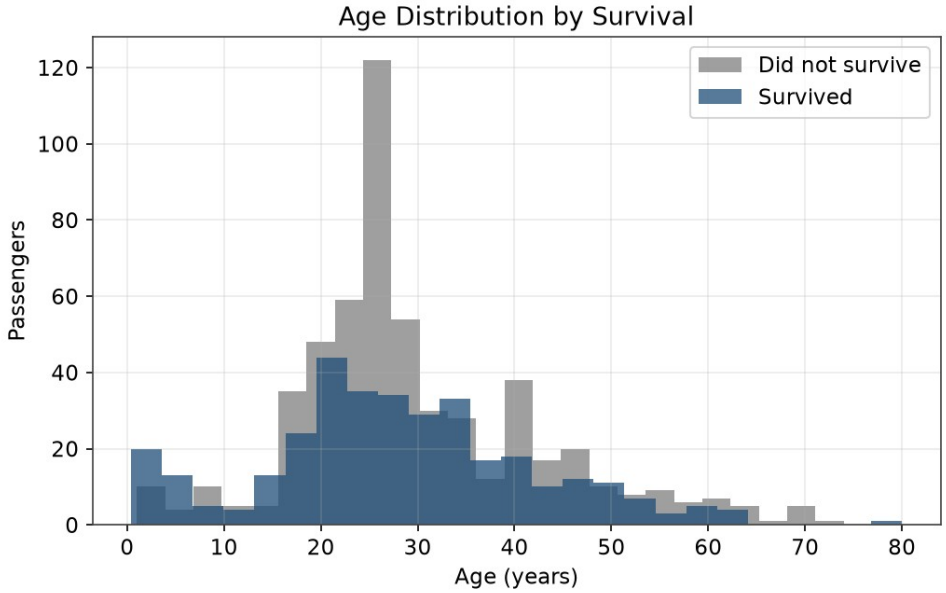


Figure 3 — Age of survivors and non-survivors.

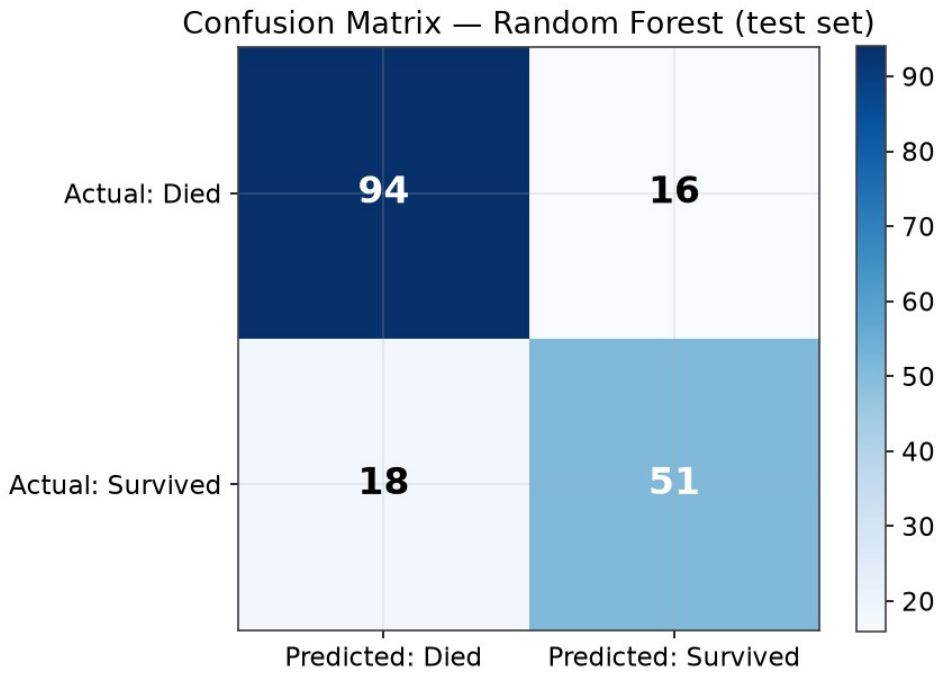


Figure 4 — Confusion matrix, Random Forest (test set).

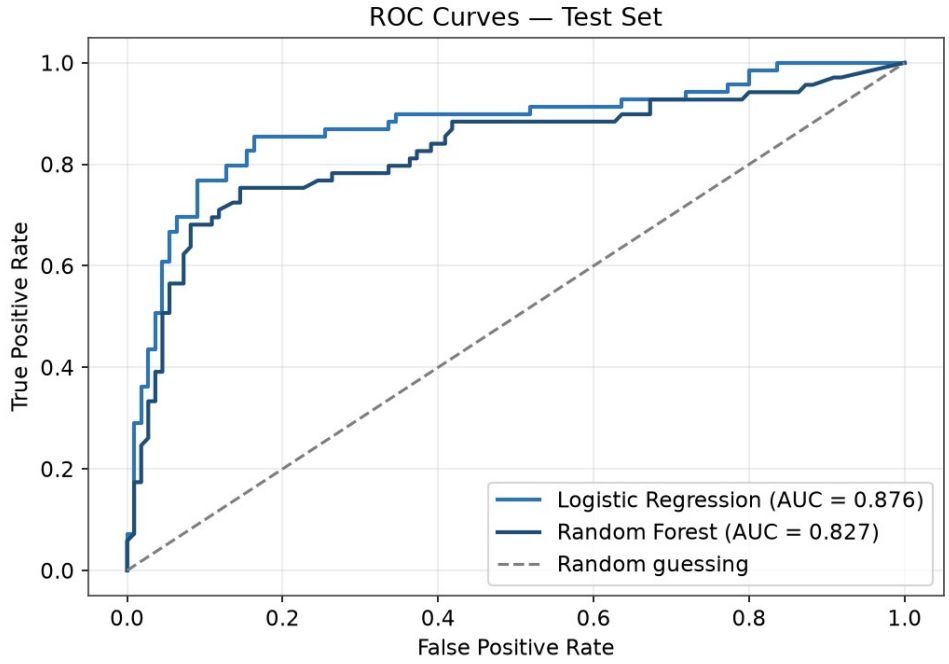


Figure 5 — ROC curves for both models (test set).

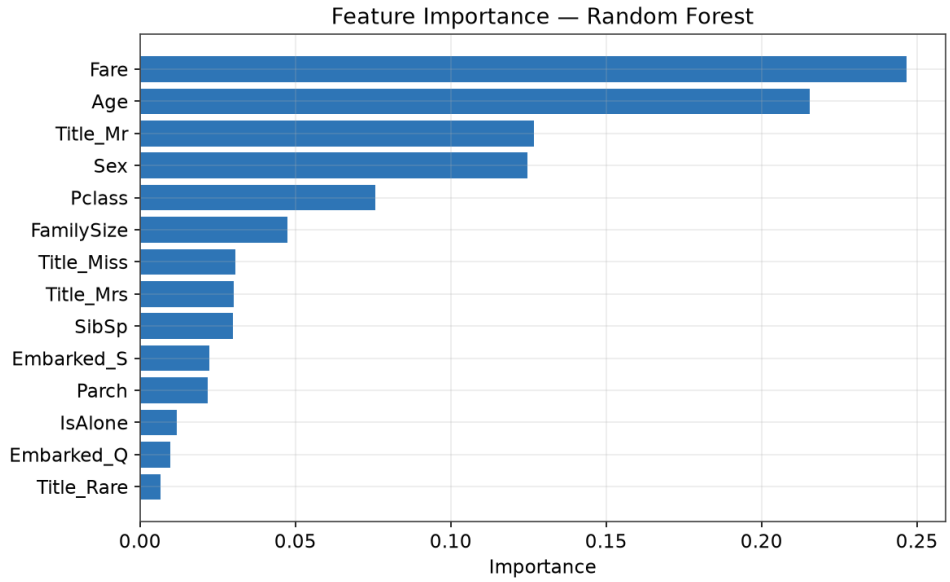


Figure 6 — Feature importance, Random Forest.

What I found:

The strongest signs were the ticket fare (0.247), age (0.216), title (0.127), sex (0.125) and ticket class (0.076). In simple words: who you were and what you paid mattered most for your chances. Family size mattered much less. The model is good (85.47% right), but it is not perfect — it still gets about 1 in 7 test passengers wrong.

Task 2 — Stock Price Prediction

My goal

I wanted to build a model that can guess the future price of a stock from its past data — open, close, high and low prices, and volume.

The data

I used Apple Inc. (AAPL) daily data from Yahoo Finance, fetched with yfinance. I tried the Stooq free CSV first, but it was blocked by a browser-verification page at fetch time. The range is 2020-01-02 to 2026-11-12: 1726 trading days. The price went from $75.09 to $354.88 in this time (+372.6%). The lowest close was $56.09 and the highest was $355.63.

How I prepared the data

I made new features from the raw prices: daily return, 7-day and 30-day moving averages of the close, the close 1 day and 5 days back, and a 7-day average of volume. That gave me 11 features.

One choice mattered more than any other. I guess the next-day return, and then I work out the price from it (price tomorrow = price today × (1 + return)). I did it this way because the price goes up a lot over the years. A tree model that learns from past prices cannot guess a price higher than any price it has seen. I tried it the direct way first. Random Forest got an R^2 of −0.43 for this exact reason. The test time went up to $355.63, but the training data never went above $259.02.

I split by time. That is the only fair way for time data. The first 80% (1,357 rows) trains the model. The last 20% (339 rows, 2025-07-10 to 2026-11-11) tests it. I never shuffled the data, so no future data leaks into training.

My code

My full code is in src/stock.py in the repo. Here are the main parts.

Making the features:

```
df["Return"] = df["Close"].pct_change()
df["MA7"] = df["Close"].rolling(7).mean()
df["MA30"] = df["Close"].rolling(30).mean()
df["Lag1"] = df["Close"].shift(1)
df["Lag5"] = df["Close"].shift(5)
df["Vol_MA7"] = df["Volume"].rolling(7).mean()
# Target: next-day return; I work out the price from it
df["TargetReturn"] = df["Close"].pct_change().shift(-1)
df["TargetPrice"] = df["Close"].shift(-1)
```

Splitting by time and training:

```
n_test = int(len(feat) * 0.20)
X_train, X_test = X.iloc[:-n_test], X.iloc[-n_test:]   # no shuffling

lr = LinearRegression()
lr.fit(StandardScaler().fit_transform(X_train), y_ret_train)

rf = RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)
rf.fit(X_train, y_ret_train)

# Work out the price from the guessed return
lr_price = close_test * (1 + lr.predict(X_test_s))
rf_price = close_test * (1 + rf.predict(X_test))
```

Checking the models against a simple baseline (tomorrow's close will be the same as today's close). Every serious guess has to beat this:

```
mean_absolute_error(y_true, y_pred)   # MAE in USD
mean_squared_error(y_true, y_pred)  # RMSE in USD
r2_score(y_true, y_pred)            # R-squared, on price and on return
```

My results

Model	MAE	RMSE	R^2 (price)	R^2 (return)	Direction
Naive baseline (tomorrow = today)	$3.12	$4.42	0.9860	0.0000	50.0%
Linear Regression	$3.59	$5.11	0.9813	-0.3127	50.4%
Random Forest	$4.82	$6.08	0.9735	-0.8725	48.7%

My honest result: neither model beat the simple baseline. The simple guess — tomorrow's price will be the same as today's — had the lowest error (MAE $3.12, RMSE $4.42). Linear Regression was close (MAE $3.59). Random Forest was worse (MAE $4.82). The R^2 on returns is below zero for both models. That means they guess daily moves worse than just guessing the average return. And they got the up/down direction right only about half the time. That is the same as a coin toss.

The high R^2 on price (0.97–0.99) looks good, but I will not quote it on its own. It is high mostly because today's price is close to tomorrow's price. It does not mean the model can predict the market. This is the result I expected for one-day prices. Daily price moves are very hard to guess from past prices alone. I do not use these models to trade, and I do not claim they can predict the market.



Figure 7 — AAPL closing price, 2020–2026, with 30-day moving average.

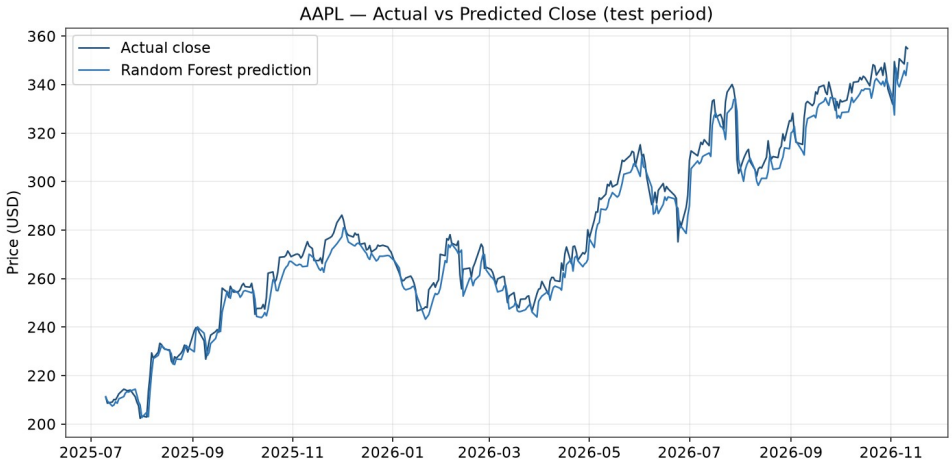


Figure 8 — Actual vs predicted close, Random Forest (test period).

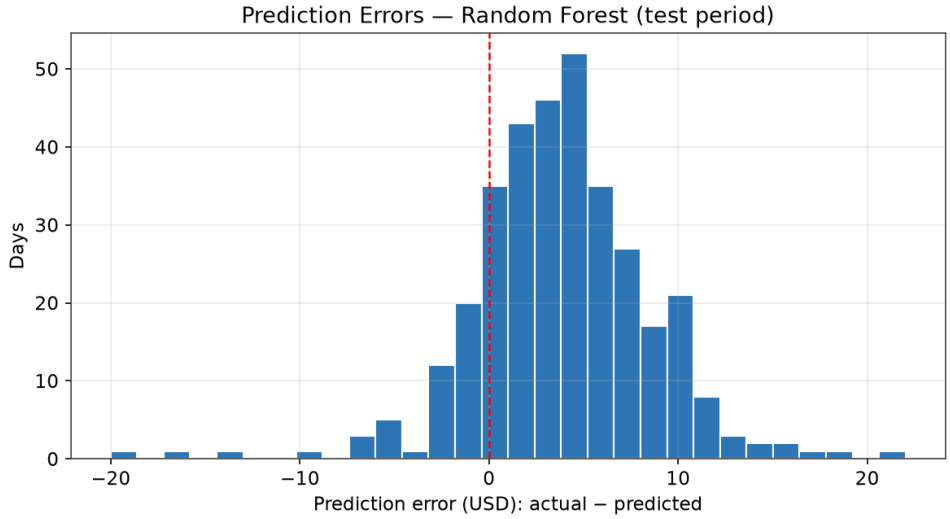


Figure 9 — Prediction errors, Random Forest (test period).

Conclusion

I finished both Month 1 tasks in full, with real data and honest checks. My Titanic model works well (85.47% right, ROC-AUC 0.876). It shows how class, sex, age and fare shaped who survived. My stock model is just as useful in the other direction. It shows, with numbers, that one-day price moves cannot be beaten by these models. That finding matters more than a nice-looking but false score would. Both tasks taught me the two main skills: how to build a model that works, and how to tell when one does not.

I will keep the same standard in Month 2: real data, code anyone can run again, and results as they are.

Tools I used

Tool	What I used it for
Python 3	My programming language
pandas	Loading, cleaning and making features
scikit-learn	Logistic Regression, Random Forest, Linear Regression, scores
matplotlib	All charts in this report
NumPy	Number work
yfinance	Apple stock data (Yahoo Finance)
python-docx	Making this report

How my work is kept: each task is its own repo, with README.md, requirements.txt, src/ (my full code), data/ (raw and cleaned CSVs) and outputs/ (metrics.json and charts). This report is in both repos under report/.